Scam Guardian: Team Plan

WHACK 26 · incident.io challenge · A real-time agent that listens to calls and protects older people from social-engineering scams

1. How the program flows

```
Microphone -> Transcriber -> Scam Brain -+-> Voice (speaks warning)
                                         +-> Dashboard (live UI)
                                         +-> Family alert
```

Each stage runs as its own **asyncio** task, and stages pass messages through queues. That is what lets three people build in parallel.

Agree these two message formats in the first hour. They are the contract between everyone's modules:

```
@dataclass
class TranscriptChunk:
    text: str
    timestamp: float
    is_final: bool        # end of a sentence/utterance

@dataclass
class Assessment:
    risk: int             # 0-100
    tactics: list[str]    # e.g. ["urgency", "safe_account", "secrecy"]
    reason: str           # short explanation
    should_speak: bool    # only interrupt if confident
    message: str          # what the agent says, in personality
```

Suggested repo layout

```
guardian/
  main.py        # wires the queues and tasks together
  models.py      # the two dataclasses above
  audio_stt.py   # Person A
  brain.py       # Person B
  voice.py       # Person B
  ui.py          # Person C
  alerts.py      # Person C
  config.py      # keys, thresholds, contact number
  samples/       # fake transcripts for testing
```

2. Who does what

Person A: ears (audio and transcription), later integration lead

- Capture mic audio with **sounddevice**
- Stream it to speech-to-text and emit **TranscriptChunk**s
- Handle the **echo problem**: when the agent speaks, the mic hears it. Ignore input while the agent is talking.
- Later, own **main.py** and get the whole pipeline running end to end

Person B: brain and personality

- Write the scam tactic list: bank "safe account", HMRC, "Hi Mum" texts, tech support, gift cards, remote-access apps, secrecy, urgency, authority
- Prompt the LLM over a rolling window of the last ~60 seconds of transcript, returning structured JSON as an **Assessment**
- Decision logic: risk thresholds, a cooldown so it doesn't repeat itself, and escalation (gentle nudge, then firm warning, then "hang up now")

- Design the personality (name, tone, example phrases) and do text-to-speech in **voice.py**

Person C: face and safety net (UI and alerts)

- Live dashboard: scrolling transcript, risk meter, highlighted tactics, agent messages
- Family alert when risk crosses a threshold
- Settings: contact number, sensitivity
- Big, simple, readable design for older users (large text, calm colours)

3. Timeline (assumes ~24 hours; scale to fit)

Phase	Time	What happens
0. Setup	Hour 0-1	**Together:** create the repo, agree models.py, get API keys working, stub every module
1. Build alone	Hours 1-6	Each person builds their module with **fake inputs**. A prints transcripts to the console, B feeds in text files from samples/, C drives the UI with fake Assessments.
2. Integrate	Hours 6-10	A leads wiring. Goal: speak into the mic and see a warning on screen and hear it. **Top priority.**
3. Improve	Hours 10-18	A: latency and echo. B: fewer false alarms, better personality. C: polish and alerts.
4. Test	Hours 18-22	Run lots of scam *and normal* calls (a normal call must not trigger it). Fix bugs. Take turns sleeping.
5. Freeze	Last 2 hours	No new features. Pitch prep only.

Rule: if nothing works end to end by hour 10, cut features until it does.

4. Resources

Libraries and services

Purpose	Recommended	Free fallback
Mic input	sounddevice	n/a
Speech-to-text	**Deepgram** streaming SDK (fast, has free credit)	faster-whisper, run locally on chunks (slower)
LLM	anthropic SDK with a fast model (Claude Haiku)	n/a
Text-to-speech	**ElevenLabs** (much better personality)	pyttsx3 (offline, robotic)
UI	**NiceGUI** (pure Python, live updates)	Streamlit (worse at real time)
Family alert	**ntfy.sh** (free push to a phone, one HTTP request)	Twilio SMS (needs an account)

Accounts and keys

- Anthropic API, Deepgram, ElevenLabs (optional). Check whether the hackathon sponsors give free credits.
- Keep keys in a **.env** file and **never commit it** (add it to .gitignore).

Hardware

- A laptop with a decent mic, a phone to make calls on speaker, and headphones for testing so the agent's voice doesn't feed back.

Content

- Common UK scam scripts for testing and for Person B's tactic list (Action Fraud and UK Finance publish descriptions).
- A few normal phone conversations to test against false alarms.

Workflow

- One GitHub repo, a branch per person, merging into **main** at least at each phase boundary.